

Brandon McFarling

Section 03

Question 1:

1. Steps:
 - a. Return sum (1 for each $\text{node.state}[i] \neq \text{self.goal}[i]$)
 - b. The minimum number of single character operations needed to fix the mismatched positions is the unavoidable remaining cost.
 - c. One move can modify at most 1 character position. If a state has k mismatched characters compared to the goal, it is impossible to reach the goal in fewer than k steps.
2. Scrabble:
 - a. Return sum ($\text{scrabble_costs}[i]$ for each $\text{node.state}[i] \neq \text{self.goal}[i]$)
 - b. The sum of the insertion costs of the exact goal letters required to replace the current mismatched positions is the unavoidable remaining cost
 - c. To reach the goal state, every mismatched position i must be changed to $\text{goal}[i]$. Even if the search takes additional intermediate steps through other letters the intermediate steps only add positive costs. Counting only the required scrabble values for destination letters means the estimate never exceeds the optimal remaining cost.
3. Frequency Cost Heuristic
 - a. $K = \text{sum}(1 \text{ for each } \text{node.state}[i] \neq \text{self.goal}[i])$
If $k == 0$ return 0 else return $k + \text{dictionary}[\text{self.goal}]$
 - b. The absolute minimum move cost for each required step is 1 plus the guaranteed rarity cost incurred when entering the goal word on the final move
 - c. The cost of any move is at least 1. My heuristic assumes every intermediate word along the path is as cheap as possible, guaranteeing the estimate stays lower or equal to actual remaining cost.

Question 2:

Word Pair	Cost Measure	UCS Cost	UCS nodes expanded	A* Cost	A* nodes expanded	Reduction
Why->Ask	Steps	5	773	5	16	97.9%
Why->Ask	Scrabble	9	765	9	13	98.3%
Why->Ask	Frequency	38.855	808	38.855	634	21.5%
Bear->Duck	Steps	4	691	4	6	99.1%
Bear->Duck	Scrabble	11	2981	11	6	99.7%
Bear->Duck	Frequency	38.240	1084	38.24	302	72.1%

1. UCS and A* always returned the same cost

2. For Why-> Ask and Bear->Duck, the scrabble cost measure produced the largest percentage reduction in expanded nodes
3. For Why->Ask and Bear->Duck, the frequency cost measure was the one where the heuristic was least informative
4. The steps cost heuristic is highly informative because every state costs exactly 1, so the heuristic provides a tight lower bound based on mismatched characters. The scrabble heuristic adds up the scrabble values of each mismatched letter. It is also highly informative as it has character specific costs, so the heuristic has lots of information on deciding what is the lowest cost path. My frequency heuristic assumes the best case rarity 0 for each intermediate state since it cannot know every intermediate state, and includes the rarity of the final goal word. This leads to a large difference in the heuristic cost and true cost, which makes it the least informative heuristic

Question 3:

Steps:

1. Yes
2. Every move costs 1, so a path forward will be the exact same cost as a path backward. This one would be a complete symmetrical cost.
3. N/A

Scrabble:

1. No
2. Action costs are asymmetric because each time you have to pay for a new letter. On the way forward, you may pay for an expensive letter, but on the way back you may pay for a cheaper one, resulting in a different cost.
3. Cat->Rat costs 1
Rat->Cat costs 3, not equal

Frequency:

1. No
2. Because the cost is based on the destination word rarity, moving from a common word to a rare word is expensive but moving from a rare word to a common word is cheap. This means the total path costs are influenced by the starting word's rarity
3. CAT->COT not equal to COT->CAT because cot and cat have different rarities

Question 4:

1. Model used: ChatGPT
Prompt: Provided prompt exactly

2. Heuristic:

A simple rarity-aware heuristic is to add a guaranteed lower bound on the rarity cost of the words that must be entered.

Let

$$d(n) = h_0(n)$$

be the number of positions in which n differs from the goal, and let

$$r_{\min} = \min_{w \in D} \text{rarity}(w),$$

where D is the supplied dictionary. Also let $r(g)$ be the rarity of the goal word g .

Define

$$h_R(n) = \begin{cases} 0, & n = g, \\ d(n) + r(g) + (d(n) - 1)r_{\min}, & n \neq g. \end{cases}$$

3. $d(n)$ = number of letters in n that differ from the goal g

$r(g)$ = rarity of the goal

$r_{\min}$ = smallest rarity of any dictionary word

$h_R(n)$ is admissible because:

- If $d(n)$ letters are wrong, it takes at least $d(n)$ steps to get to the goal state
 - Every move costs at least 1 point
 - The heuristic assumes the lowest possible rarity for each intermediate word
- This means that the heuristic assumes the best case scenario for each step, so the heuristic cost will be less than or equal to the actual optimal cost
- Also, the $h_R(n)$ is never smaller than $h_0(n)$, since $d(n) \geq 1$ and $r(g) \geq 0$ and $r_{\min} \geq 0$:

$$h_R(n) = d(n) + r(g) + (d(n) - 1)r_{\min} \geq d(n) = h_0(n)$$

And $h_R(n)$ is larger than $h_0(n)$ for at least one state:

$$h_R(n) - h_0(n) = r(g) + (d(n) - 1)r_{\min} \quad (\text{This is just } h_R(n) - d(n), \text{ and } d(n) = h_0(n))$$

4. N/A

5.

Word Pair	Heuristic	UCS cost	UCS nodes expanded	Heuristic Cost	Heuristic nodes expanded	Reduction
Why->Ask	Baseline $h_0(n)$	38.855	808	38.855	755	6.56%

Why->Ask	$hR(n)$	38.855	808	38.855	634	21.5%
Bear->Duck	Baseline $h_0(n)$	38.240	1084	38.240	720	33.58%
Bear->Duck	$hR(n)$	38.240	1084	38.240	302	72.1%

The experimental evidence supports the use of the proposed heuristic because the reduction by $hR(n)$ is better than the baseline heuristic. $hR(n)$ adds $r(g)$ to non goal states, which breaks ties in favor of the goal. This allows for a higher reduction of cost as shown by the experiments. Because $r_{min} = 0$ for the intermediate steps, the nodes expanded match exactly with my own proposed heuristic in the earlier problems. Both assume the safest admissible bound for $r(g)$, and both are better than the baseline heuristic.